Disjoint Set

Introduction

Disjoint sets in mathematics are two sets that don't have any element in common. Sets can contain any number of elements, and those elements can be of any type. We can have a set of cars, a set of integers, a set of colors, etc. Sets also have various operations that can be performed on them, such as union, intersection, difference, etc.

We focus on **Disjoint Set Data Structure** and the operations we can perform. we will go through what a disjoint set data structure is, the disjoint set operations, as well as examples.

Disjoint Set Data Structure

A disjoint set data structure is a data structure that stores a list of disjoint sets. In other words, this data structure divides a set into multiple subsets - such that no 2 subsets contain any common element. They are also called union-find data structures or merge-find sets.

For example: if the initial set is [1,2,3,4,5,6,7,8].

A Disjoint Set Data Structure might partition it as - [(1,2,4), (3,5), (6,8),(7)].

This contains all of the elements of the original set, and no 2 subsets have any element in common.

The following partitions would be invalid:

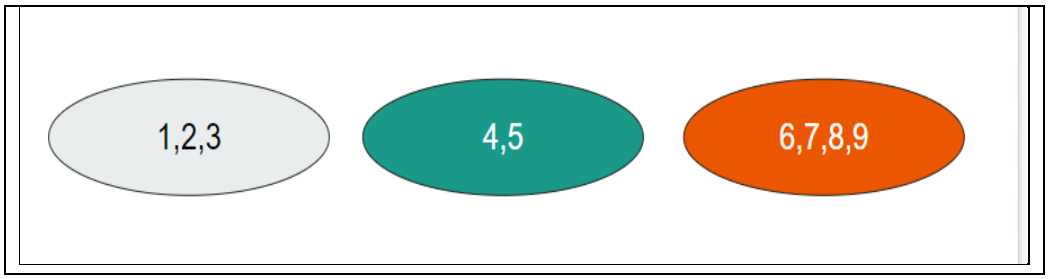
[(1,2,3),(3,4,5),(6,8),(7)] - invalid because 3 occurs in 2 subsets.

[(1,2,3),(5,6),(7,8)] -invalid as 4 is missing.

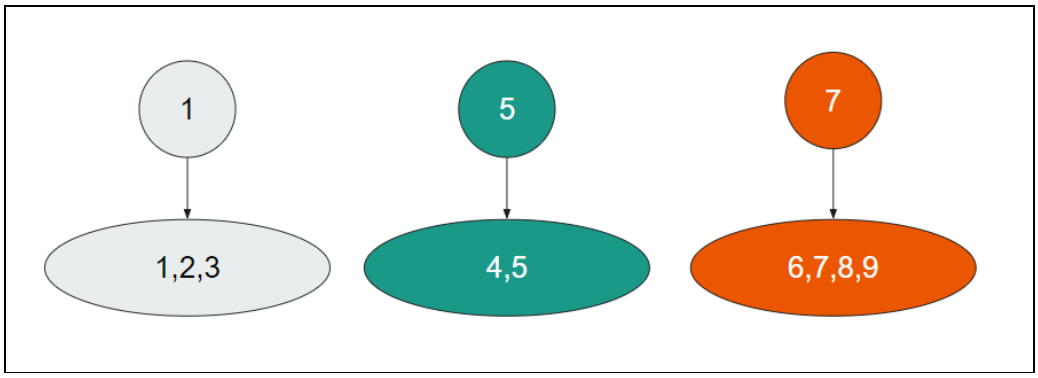
Representative Member

Each subset in a Disjoint Set Data Structure has a **representative member**. More commonly, we call this the **parent.** This is simply the member *responsible for identifying* the subset.

Let's understand this with an example.
Suppose this is the partitioning of sets in our Disjoint Set Data structure.



We wish to identify each subset, so we assign each subset a value by which we can identify it. The easiest way to do this is to choose a **member** of the subset and make it the **representative member**. This representative member will become the **parent** of all values in the subset.



Here, we have assigned representative members to each subset. Thus, if we wish to know which subset 3 is a part of, we can simply say - 3 is a part of the subset with representative member 1. More simply, we can say that the parent of 3 is 1.

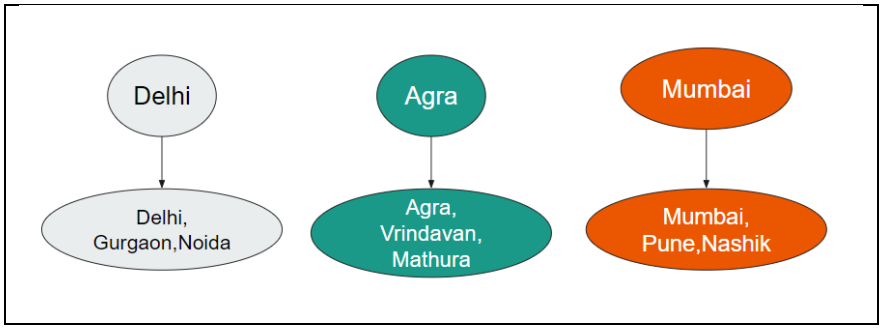
Here, 1 is its own parent.

Disjoint Set Operations

There are 2 major disjoint set operations in daa:

1. **Union/Merge** - this is used to merge 2 subsets into a single subset.
2. **Find** - This is used to find which subset a particular value belongs to.

We will take a look at both of them. To make things easier, we will change our sets from integers to Cities.

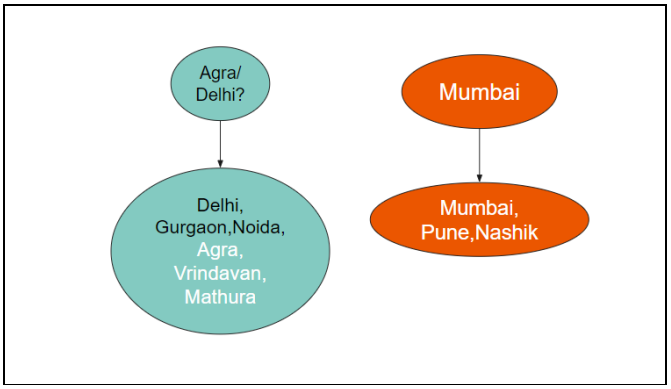


Let's say this data structure represents the places connected via express trains/ metro lines or any other means.

Union/Merge:

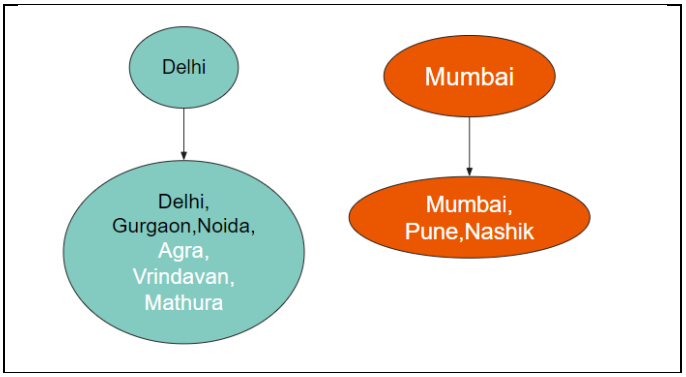
Suppose, in our example - a new express train started from Delhi to Agra. This would mean that *all* the cities connected to Delhi are now connected to *all* the cities connected to Agra.

This simply means that we can ***merge*** the subsets of Agra and Delhi into a single subset. It will look like this.



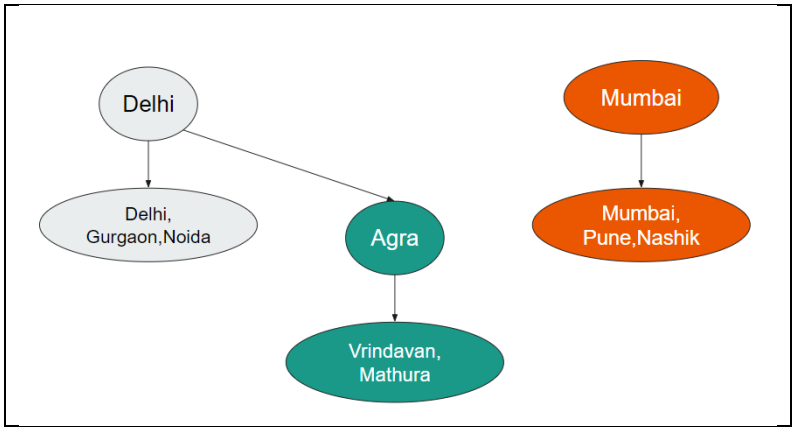
One key issue here is which out of Agra and Delhi will be the new Parent? For now, we will just choose any of them, as it will be sufficient for our purposes. Let's take Delhi for now.

Further optimizations can be made to this by choosing the parent based on which subset has a larger number of values.



This is the new state of our Disjoint Set Data Structure. Delhi is the parent of Delhi, Gurgaon, Noida, Agra, Vrindavan, and Mathura.

This would mean that we would have to change the parent of Agra, Vrindavan, and Mathura (basically, all the children of Agra), to Delhi. This would be linear in time. If we perform the union operation m times, and each union takes O(n) times, we would get a quadratic O(mn) time complexity. This is not optimized enough.
Instead, we structure it like this:



Earlier, Agra was its own parent. Now, Delhi is the parent of Agra.
Now, you might be wondering - The parent of Vrindavan is still Agra. It should have been Delhi now.
This is where the next operation helps us - the **find** operation.

Find:

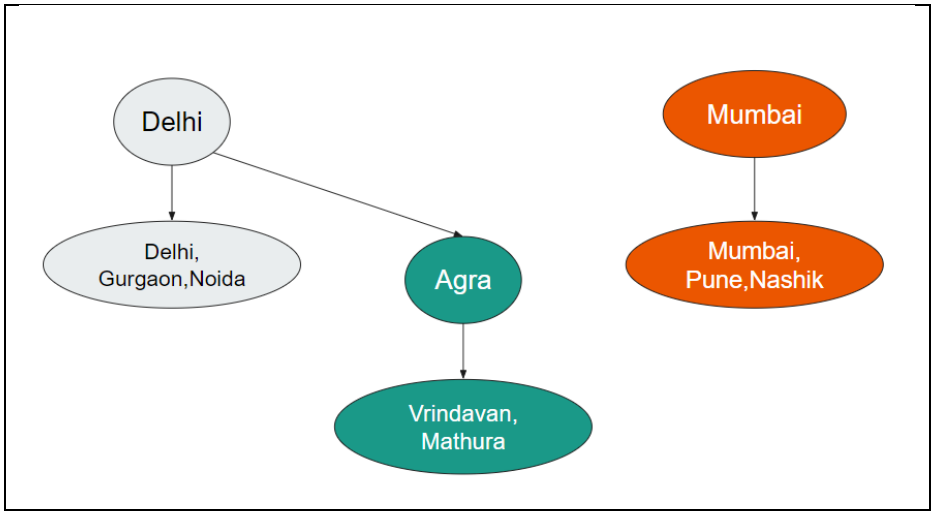
The find operation helps us find the parent of a node. As we saw above, the direct parent of a node might not be its actual (logical) parent. E.g., the logical parent of Vrindavan should be Delhi in the above example. But its direct parent is Agra.

So, how do we find the actual parent?

The find operation helps us to find the actual parent. In pseudocode, the find operation looks like this:

```
find(node):
if (parent(node)==node) return node;
else return find(parent(node));
```

We don't return the direct parent of the node. We keep going up the tree of parents until we find a node that is its own parent.
Let's understand this via our example.

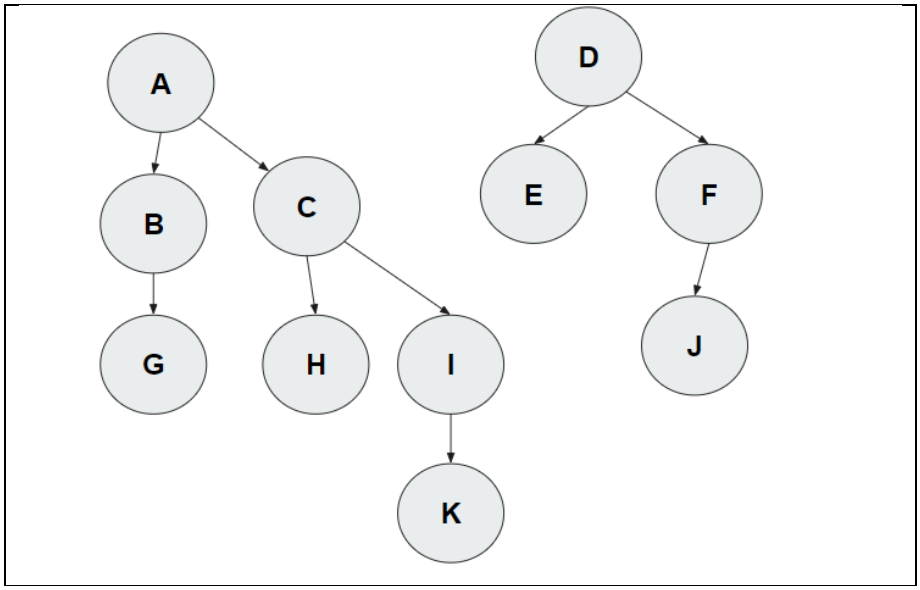


Suppose we have to find the parent of Mathura.

1. Check the direct parent of Mathura. It's Agra. Is Agra == Mathura?
   The answer is false. So, now we call the find operation on Agra.
2. Check the direct parent of Agra. It's Delhi. Is Delhi ==Agra?
   The answer is false. So now we call the find operation on Delhi.
3. Check the direct parent of Delhi. It's Delhi. Is Delhi==Delhi.
   The answer is true! So, our final answer for the parent of Mathura is Delhi.

This way, we keep going "up" the tree until we reach a root element. A root element is a node with its own parent - in our case, Delhi.

Example:



Let's try to find the parent of K in this example. (A and D are their own parents)

1. ParentOf(K) is I. Is I == K?
   False. So, we move upward, now using I.
2. ParentOf(I) is C. Is C== I?
   False. Move upward using C.
3. ParentOf(C) is A. Is A==C?
   False. Move upward using A.
4. ParentOf(A) is A. Is A==A?
   True! Our final answer is A